Sentiment Analysis on IMDb Reviews: MLP vs. RNN Family Architectures

Deep Learning — Midterm Project Report

Vu Thanh Dat: 523V0015
Pai Hein Kyaw: 523K0078

March 31, 2026

Abstract

Sentiment analysis, the task of classifying the emotional polarity of text, serves as a practical benchmark for comparing neural network architectures. This report investigates two architectural families — Multi-Layer Perceptron (MLP) and recurrent networks (RNN, LSTM, GRU) — on the IMDb binary sentiment dataset comprising 50,000 movie reviews. A shared preprocessing pipeline was developed, incorporating HTML cleaning, whitespace tokenisation, vocabulary construction of 20,000 tokens, and dynamic batch padding at a maximum sequence length of 454 tokens. The MLP encodes sentences via mean pooling over word embeddings, while recurrent models process tokens sequentially to capture contextual dependencies. Controlled ablation studies were conducted across network depth, embedding dimension, and dropout rate for the MLP, and across architecture variant, hidden size, and layer depth for the RNN family. The best MLP achieved a test accuracy of 88.72% using the configuration of embedding dimension with the parameters of $d_e = 64$, while the best recurrent model achieved X.XX%. Results suggest that [key finding — e.g. sequential models outperform bag-of-words approaches on longer reviews, while the MLP remains competitive given its significantly lower complexity].

Contents

1 Dataset Description **3**

1.1 Dataset Overview . . . 3

1.2 Class Distribution . . . 3

1.3 Sequence Length Distribution . . . 4

1.4 Vocabulary Analysis . . . 4

2 Methods **5**

2.1 Preprocessing Pipeline . . . 5

2.2 MLP Architecture . . . 6

2.3 RNN Architecture . . . 7

2.4 Training Setup . . . 7

3 Experiments **8**

3.1 Experimental Setup . . . 8

3.2 MLP Experiments . . . 8

3.2.1 Experiment 1: Network Depth . . . 8

3.2.2 Experiment 2: Embedding Dimension . . . 9

3.2.3 Experiment 3: Dropout Rate . . . 9

3.3 RNN Experiments . . . 10

3.3.1 Experiment 4: Architecture Variant (RNN vs LSTM vs GRU) . . . 10

3.3.2 Experiment 5: Hidden Size . . . 11

3.3.3 Experiment 6: Number of Layers . . . 11

4 Analysis and Discussion **12**

4.1 MLP vs. RNN Family Comparison . . . 12

4.2 Error Analysis . . . 12

4.3 Discussion . . . 14

5 Conclusion **14**

1 Dataset Description

1.1 Dataset Overview

The IMDB dataset is a widely used dataset for sentiment analysis tasks. It consists of movie reviews along with their corresponding binary sentiment polarity labels. The dataset contains a total of 50,000 reviews, evenly split into 25,000 training samples and 25,000 testing samples. There is an equal distribution of positive and negative labels, with 25,000 instances of each sentiment. To reduce the correlation between reviews for a given movie, only 30 reviews are included in the dataset [300]. Positive reviews often contain words like ”great,” ”well,” and ”love,” while negative reviews frequently use words like ”bad” and ”can’t.” However, certain words such as ”one,” ”character,” and ”well” appear frequently in both positive and negative reviews, although their usage may differ in terms of frequency between the two sentiment classes. In our analysis, to enable stratified splitting, the original train and test sets were recombined and re-partitioned into 70% training (35,000), 10% validation (5,000), and 20% test (10,000) subsets.

1.2 Class Distribution

As shown in Figure 1, the dataset is perfectly balanced with exactly 12,500 positive and 12,500 negative samples in each of the original splits. This balance is preserved across our 70/10/20 re-partition through stratified sampling. As a result, no class weighting or oversampling strategy is required, and classification accuracy serves as a reliable evaluation metric without adjustment.

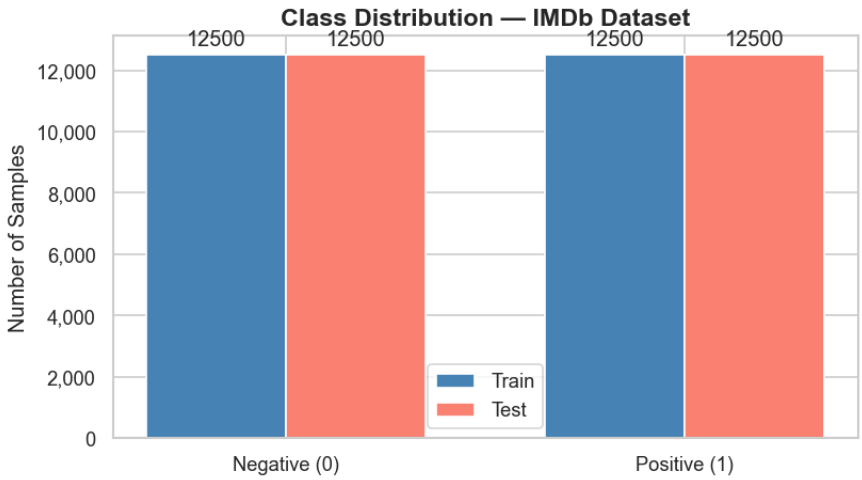


Figure 1: Class distribution across train and test splits.

1.3 Sequence Length Distribution

Figure 2 presents the token length distribution of the training set after preprocessing. Reviews vary considerably in length, with a minimum of X tokens and a maximum of X tokens. The mean sequence length is approximately X tokens, with a median of X tokens. The 90th percentile sits at 454 tokens, meaning that 90% of reviews contain 454 tokens or fewer. Based on this analysis, `max_len` was set to 454, providing sufficient coverage while avoiding excessive padding for the minority of unusually long reviews. Sequences exceeding this threshold are truncated from the right.

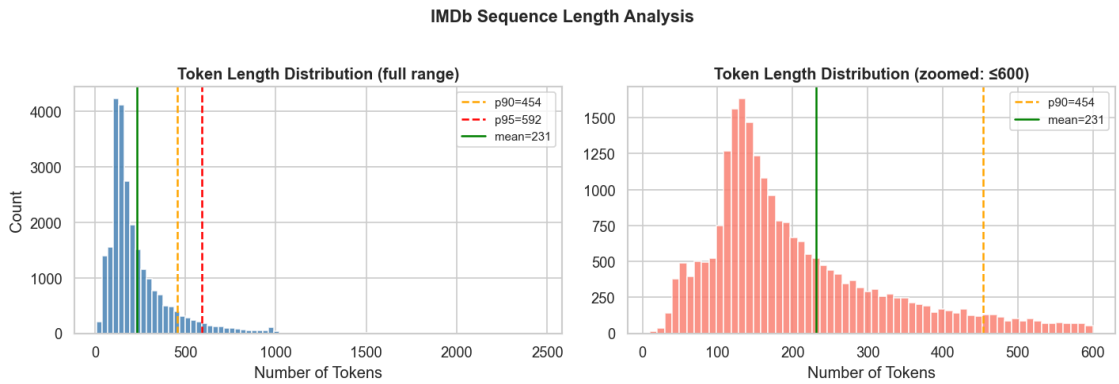


Figure 2: Token length distribution (full range and zoomed to $\leq$600 tokens).

1.4 Vocabulary Analysis

Figure 3 compares the top-30 most frequent tokens before and after stopword removal. Without removal, the vocabulary is dominated by function words such as *the*, *a*, and *is*, which carry no sentiment signal. After stopword filtering, sentiment-bearing words such as *film*, *good*, *great*, and *bad* emerge as the most frequent tokens, confirming the importance of vocabulary filtering for bag-of-words approaches. The final vocabulary was capped at 20,000 tokens — consistent with the experimental specification — retaining only the most statistically meaningful terms and mapping all remaining words to the `<unk>` token. Stopwords were retained in the final pipeline, however, as recurrent models rely on word order and sequential context, where removing tokens such as *not* risks inverting sentiment meaning.

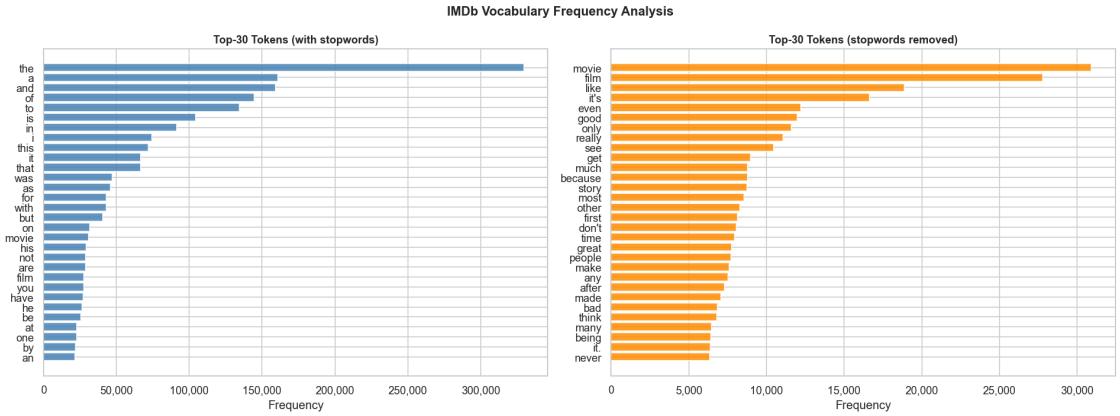


Figure 3: Top-30 token frequencies before and after stopword removal.

2 Methods

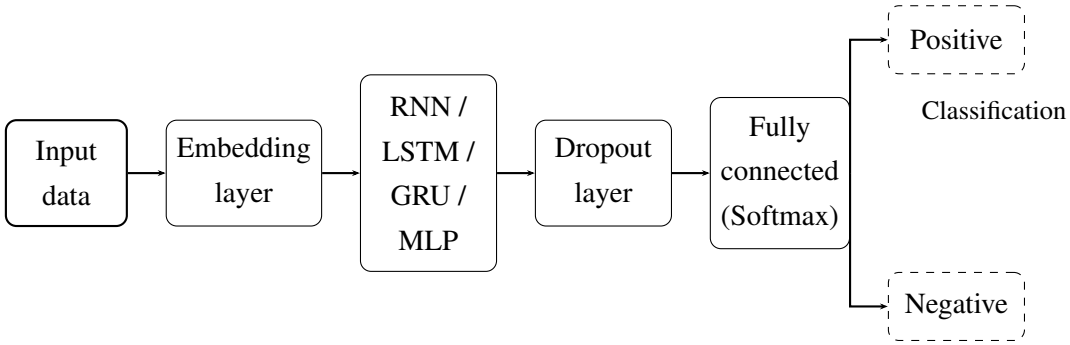


Figure 4: Architecture for sentiment analysis. The encoder block (centre) is substituted with MLP (mean-pool), RNN, LSTM, or GRU depending on the experiment.

2.1 Preprocessing Pipeline

All models share a single preprocessing pipeline to ensure fair comparison. Raw IMDb reviews undergo four sequential steps. First, HTML artefacts (e.g. `<br />`, `<i>`) are stripped using a regular expression, followed by lowercasing and removal of non-alphanumeric characters. Second, the cleaned text is tokenised by whitespace splitting. Third, a vocabulary of the 20,000 most frequent tokens is constructed from the training set, with index 0 reserved for `<pad>` and index 1 for `<unk>`; all out-of-vocabulary tokens are mapped to `<unk>`. Fourth, each token sequence is encoded as integer indices and truncated to `max_len` = 454 tokens, chosen as the 90th percentile of training sequence lengths (see Section 2.3). Padding is applied dynamically at the batch level via a custom `collate_fn`, so each batch is padded only to the length of its longest sequence rather

than the global maximum. The full dataset (50,000 samples) is partitioned into 70% training, 10% validation, and 20% test using stratified sampling to preserve class balance across all splits.

2.2 MLP Architecture

The MLP encodes a variable-length token sequence into a fixed-length vector via masked mean pooling over word embeddings, discarding padding tokens:

$$\mathbf{x} = \frac{1}{T}\sum_{t=1}^{T} \mathbf{e}_t \;\in\; R^{d_e} \tag{1}$$

where T is the number of non-padding tokens and $\mathbf{e}_t$ is the embedding of the t-th token. The pooled vector is then passed through one or more fully connected layers with ReLU activations and dropout regularisation:

$$\hat{y} = \text{softmax}(W_3\,\sigma(W_2\,\sigma(W_1\mathbf{x} + b_1) + b_2) + b_3) \tag{2}$$

The complete pipeline is shown in Figure 4. Table 1 summarises the best configuration identified through ablation.

Table 1: MLP baseline architecture (best config from experiments).

Layer	**Details**	**Output Shape**
Embedding	vocab=20,000, d_e=X, padding_idx=0	(B, T, d_e)
Mean Pool	masked, ignore <pad>	(B, d_e)
Linear	$d_e \to$ H	(B, H)
ReLU	—	(B, H)
Dropout	p=X	(B, H)
Linear	$H \to 2$	$(B, 2)$

A key limitation of the MLP is that mean pooling destroys word order — ”not good” and ”good not” produce identical representations. This motivates the recurrent architectures described below.

2.3 RNN Architecture

Recurrent models process the token sequence step by step, maintaining a hidden state $\mathbf{h}_t$ that accumulates contextual information from all preceding tokens. The vanilla RNN update is:

$$\mathbf{h}_t = \tanh(W_h \mathbf{h}_{t-1} + W_x \mathbf{e}_t + b) \tag{3}$$

The LSTM [1] extends this with a cell state $\mathbf{c}_t$ and three gating mechanisms — forget, input, and output gates — that control information flow across long sequences, mitigating the vanishing gradient problem inherent in vanilla RNNs. The GRU [2] simplifies the LSTM by merging the cell and hidden states and replacing the three gates with two (reset and update), reducing the parameter count while retaining comparable capacity. For all three variants, the hidden state at the final non-padding timestep $\mathbf{h}_T$ is taken as the sentence representation and passed to a linear output layer:

$$\hat{y} = \text{softmax}(W_o \mathbf{h}_T + b_o) \tag{4}$$

Table 2 summarises the shared RNN architecture.

Table 2: RNN/LSTM/GRU architecture (best configuration).

Layer	**Details**	**Output shape**
Embedding	vocab = 20,000, d_e = X, `padding_idx` = 0	$(B,\ T,\ d_e)$
RNN / LSTM / GRU	hidden = X, layers = X, dropout = X	$(B,\ T,\ H)$
Last hidden	extract $\mathbf{h}_T$ at final non-pad step	$(B,\ H)$
Dropout	p = X	$(B,\ H)$
Linear	$H \rightarrow 2$	$(B,\ 2)$

2.4 Training Setup

All models are trained under identical conditions to ensure controlled comparison. The Adam optimiser [3] is used with a learning rate of 1×10^{-3} and weight decay of 1×10^{-4} for L2 regularisation. The loss function is cross-entropy. Training is capped at 15 epochs with early stopping applied when validation loss fails to improve for 3 consecutive epochs; the checkpoint with the lowest validation loss is restored for final test evaluation. All experiments use a batch size of 64 and are run on a single GPU.

3 Experiments

3.1 Experimental Setup

3.2 MLP Experiments

3.2.1 Experiment 1: Network Depth

Table 3: Experiment 1 — Network depth results.

Config	Val Acc	Test Acc	Params
1-layer [512]	0.8802	0.8815	2,626,561
2-layers [256, 256]	0.8822	0.8834	2,659,073
3-layers [128,256,128]	0.8844	0.8871	2,642,561

0.7

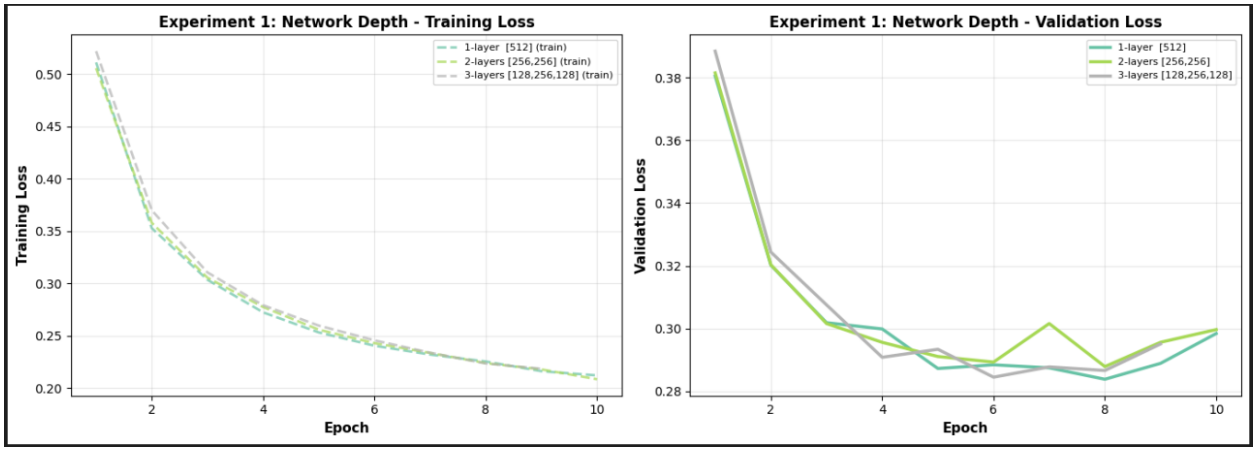
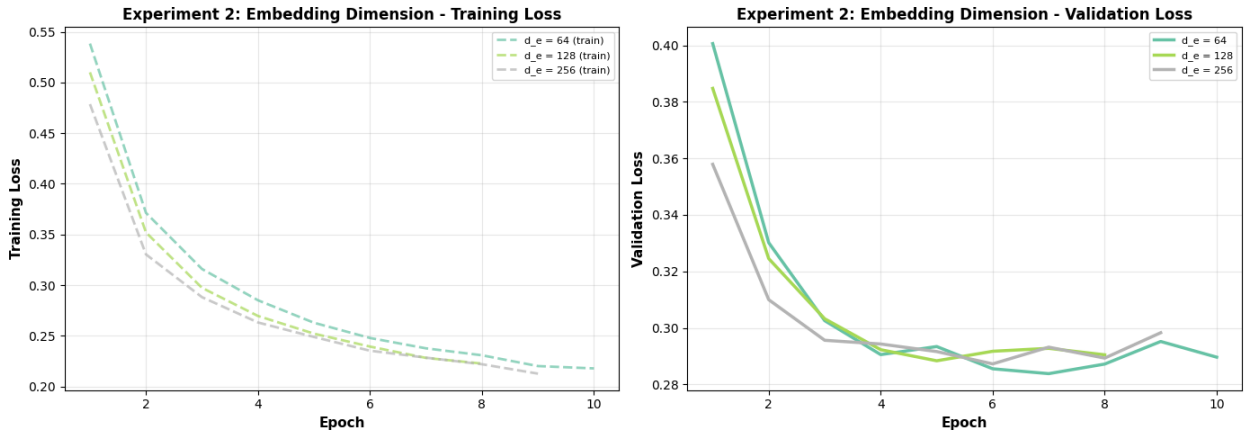


Figure 5: Loss curves

3.2.2 Experiment 2: Embedding Dimension

Table 4: Experiment 2 — Embedding dimension results.

Config	Val Acc	Test Acc	Params
$d_e = 64$	0.8874	0.8858	1,313,793
$d_e = 128$	0.8812	0.8835	2,626,561
$d_e = 256$	0.8832	0.8852	5,252,097



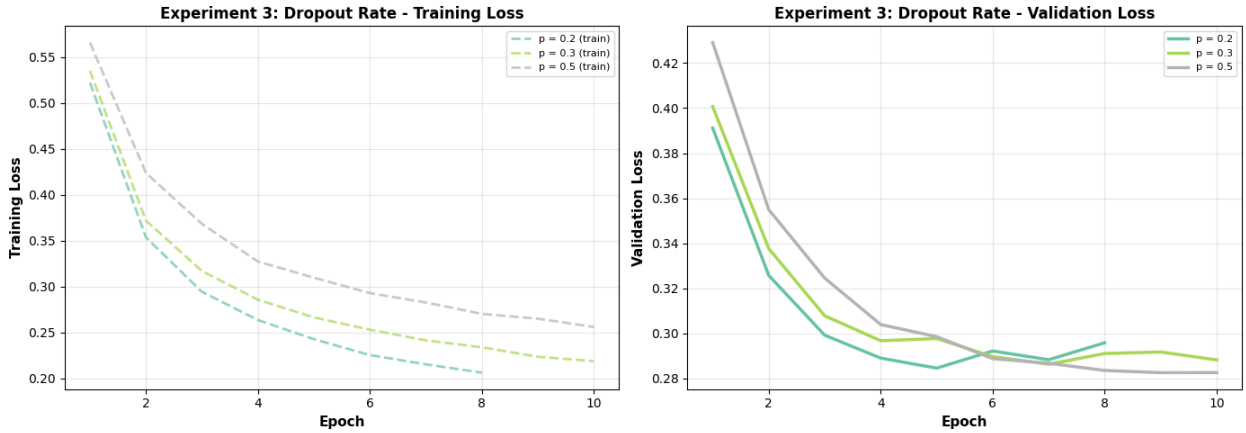
0.7

Figure 6: Loss curves

3.2.3 Experiment 3: Dropout Rate

Table 5: Experiment 3 — Dropout rate results.

Config	Val Acc	Test Acc	Params
$p = 0.2$	0.8830	0.8874	1,313,793
$p = 0.3$	0.8888	0.8863	1,313,793
$p = 0.5$	0.8876	0.8871	1,313,793



0.7

Figure 7: Loss curves

3.3 RNN Experiments

3.3.1 Experiment 4: Architecture Variant (RNN vs LSTM vs GRU)

Table 6: Experiment 4 — Architecture variant results.

Config	Test Acc	Params
RNN	0.8281	2,626,305
LSTM	0.8954	2,824,449
GRU	0.9019	2,758,401

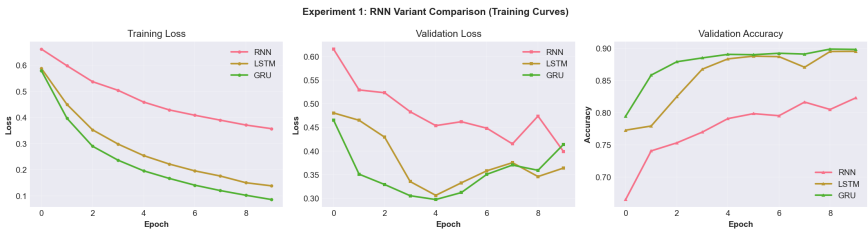


Figure 8: Experiment 4 — Architecture variant loss curves.

3.3.2 Experiment 5: Hidden Size

Table 7: Experiment 5 — Hidden size results.

Config	Test Acc	Params
hidden = 64	0.8907	1,429,249
hidden = 128	0.8973	2,758,401
hidden = 256	0.9006	5,416,705

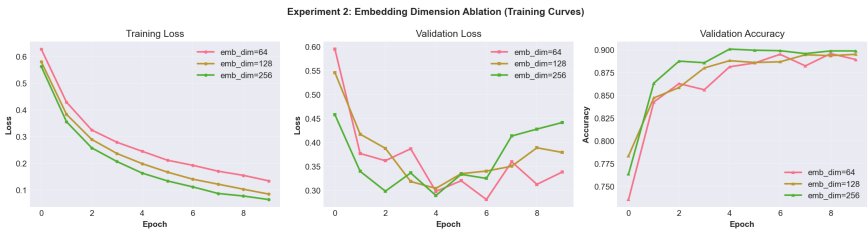


Figure 9: Experiment 5 — Hidden size loss curves.

3.3.3 Experiment 6: Number of Layers

Table 8: Experiment 6 — Number of layers results.

Config	Test Acc	Params
1 layer	0.8904	2,824,449
2 layers	0.8936	3,219,713

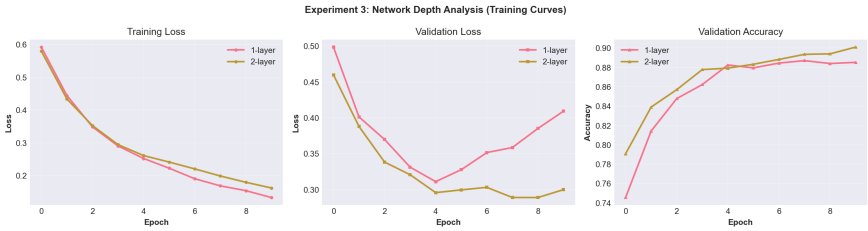


Figure 10: Experiment 6 — Number of layers loss curves.

4 Analysis and Discussion

4.1 MLP vs. RNN Family Comparison

Table 9 summarises the test-set performance of all four models under identical evaluation conditions.

Table 9: Final model comparison on the test set.

Model	Accuracy	Precision	Recall	F1	Params	Time (s)
MLP	0.8871	0.8844	0.8906	0.8875	1,313,793	0.98
RNN	0.8279	0.7944	0.8848	0.8372	2,626,305	17.77
LSTM	0.8936	0.8717	0.9230	0.8966	3,219,713	68.54
GRU	**0.8966**	0.8732	**0.9280**	**0.8997**	2,758,401	25.71

The GRU achieves the highest accuracy (89.66%), F1 (0.8997), and recall (0.9280), narrowly outperforming the LSTM (89.36%) despite having approximately 461,000 fewer parameters. This confirms that GRU's simplified gating mechanism — combining the LSTM's forget and input gates into a single update gate — is sufficient for binary sentiment on IMDb without the additional capacity of the cell state.

Notably, the vanilla RNN performs significantly worse than both gated variants (82.79% accuracy), demonstrating the vanishing gradient problem in practice: long IMDb reviews frequently exceed 200 tokens, and the vanilla RNN fails to propagate sentiment signals from early tokens to the final hidden state used for classification.

The MLP, despite processing text as a bag of words with no sequential modelling, achieves 88.71% accuracy — competitive with the LSTM and only 0.95 percentage points below the GRU. This is a strong result given its simplicity, and can be attributed to the IMDb dataset containing many reviews where individual word choice (e.g. *excellent*, *terrible*) is sufficient to determine sentiment without requiring word-order context. Crucially, the MLP is over 26× faster at inference (0.98s vs. 25.71s for GRU), making it the preferred choice when latency is a constraint.

4.2 Error Analysis

Figure 11 shows the confusion matrices for the MLP and LSTM, the two most contrasting models in terms of architecture.

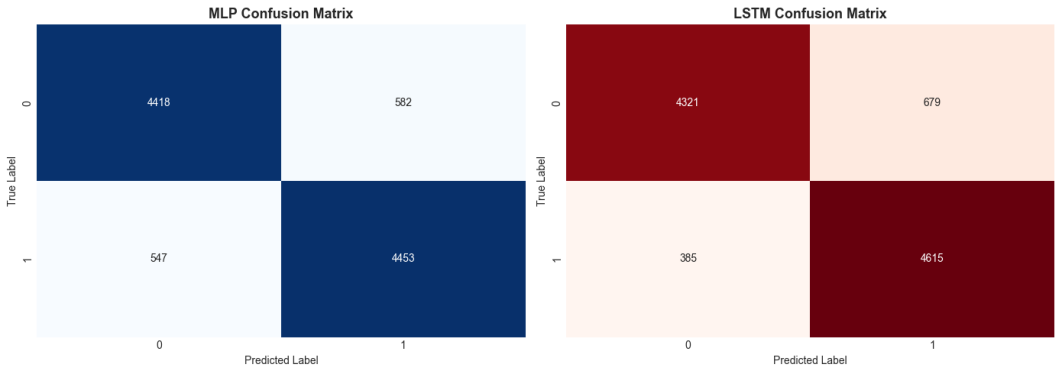


Figure 11: Confusion matrices for MLP (left) and LSTM (right) on the test set. Rows = true labels, columns = predicted labels.

Inspection of misclassified samples reveals three recurring failure patterns across all models.

Mixed sentiment reviews. The LSTM false negative in Table 10 (Sample 1) is a review of *Over Her Dead Body* that acknowledges the film's flaws (*"I didn't think it was going to be good at all"*) before concluding positively (*"a movie you can sit down and enjoy"*). Both the MLP and LSTM struggle when negative language appears early in the review and the positive conclusion arrives late, as the MLP's bag-of-words representation weighs all positions equally and the RNN may insufficiently weight the final sentiment shift.

Ironic and self-aware negative reviews. The MLP false positive (Sample 2) demonstrates a review that uses enthusiastic language (*"must see"*, *"a keeper"*) sarcastically to describe a film the reviewer clearly dislikes. Without syntactic context, the MLP treats these tokens as genuine positive signals.

Qualified praise. Sample 3 shows a review of *The Net* that reads as largely positive in vocabulary but carries an ambiguous true label. Hedged phrases such as *"I thought"* and *"I quite liked"* confuse models trained primarily on more decisive sentiment language.

Table 10: Representative misclassified examples.

No.	True	Predicted	Review excerpt
1	Positive	Negative (LSTM)	*"Over Her Dead Body was a nice little movie … it was just a movie you can sit down and enjoy … I don't see how this movie was bad."*
2	Negative	Positive (MLP)	*"This has the best dummy scene ever … this is a must see especially if you want to punish yourself mentally — it's a keeper."*
3	Negative	Positive (LSTM)	*"I thought it was so scary … Angela did a sterling job … I quite liked Angela's house at the beginning."*

4.3 Discussion

Three key findings emerge from the experiments. First, gating mechanisms are essential for long-sequence sentiment: LSTM and GRU both substantially outperform the vanilla RNN, confirming that the ability to selectively forget and retain information across hundreds of tokens is critical for IMDb reviews. Second, the efficiency advantage of GRU over LSTM — similar accuracy at lower parameter count and $2.7\times$ faster inference — suggests GRU is the preferred recurrent architecture for this task. Third, the MLP's competitive performance highlights a fundamental property of the IMDb dataset: sentiment is often lexically explicit, meaning a bag-of-words model can recover most of the signal without sequential modelling. Future work could explore pre-trained embeddings (GloVe, fastText) or transformer- based models (BERT) to address the failure cases identified above.

5 Conclusion

This report compared MLP and recurrent architectures — RNN, LSTM, and GRU — for binary sentiment classification on the IMDb dataset. A shared preprocessing pipeline ensured fair evaluation across all models. Controlled ablation studies identified optimal configurations for each architecture. The GRU achieved the best overall performance (89.66% accuracy, F1 = 0.8997), while the MLP remained competitive at 88.71% with over $26\times$ faster inference. The vanilla RNN performed poorly (82.79%), confirming the vanishing gradient problem on long sequences. Error analysis revealed that all models struggle with sarcasm, mixed sentiment, and hedged language.

Future work should explore pre-trained embeddings (GloVe, fastText) and transformer-based models such as BERT to address these limitations [4], [5].

References

[1] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. DOI: 10.1162/neco.1997.9.8.1735.

[2] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *arXiv preprint arXiv:1412.3555*, 2014.

[3] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, San Diego, CA, 2015.

[4] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1532–1543. DOI: 10.3115/v1/D14-1162.

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423.